title	Problem Statement	techstack
CodeSense	Take a short code snippet as input and return an English explanation using a text-generation model.	MERN with API call to ML services
MindMend	Generate positive affirmations based on the user's daily journal sentiment.	MERN with API call to ML services
VibeBeats	Detects your vibe from text and drops mood-matching song recommendations.	MERN with API call to ML services
LinguaLink	Lets users chat in any language with instant translation through AI.	MERN with API call to ML services
SoulSync	An AI buddy that listens to your thoughts and responds with calm, supportive messages to help you feel better.	MERN with API call to ML services
Image Library with Auto-Captions	Create an image library where uploads trigger asynchronous caption generation. API stores metadata; RabbitMQ brokers Celery jobs; workers call external captioning models. Redis caches recent captions; MongoDB stores metadata and tags. Logstash collects logs; Prometheus tracks queue depth and durations.	Frontend: Upload UI with preview and status. Node.js + Express: JWT auth, uploads, job publish. RabbitMQ: Message broker providing durable queues/back-pressure. Celery (Python): Runs captioning tasks. Redis: Caches job status and recent captions. MongoDB: Stores image metadata, captions, tags. Prometheus: Queue depth, task duration, success ratio. Logstash: Central logs (upload events, task outcomes, errors). External AI APIs (examples): Hugging Face: nlpconnect/vit-gpt2-image-captioning — Image caption generation endpoint
Voice Journal (Speech-to-Text)	Implement a voice journal that converts audio uploads to text asynchronously. Celery uses Redis as broker+results, calls FastAPI for STT which may proxy to external Whisper models. MongoDB stores transcripts; Redis caches recent items. Prometheus tracks latency; Logstash centralizes logs.	Frontend: Audio upload and transcript viewer. Node.js + Express: JWT, REST endpoints, enqueuing. Redis: Broker + result backend and cache. Celery (Python): STT jobs, retries, backoff. FastAPI (Python): Internal STT gateway to external model. MongoDB: Stores transcripts and metadata. Prometheus: Latency and throughput for STT pipeline. Logstash: Central logs with traceId/jobId for debugging. External AI APIs (examples): Hugging Face: openai/whisper-small — Speech-to-text endpoint.
Reading List with AI Summaries	Create a reading list where URLs are scraped and summarized asynchronously. Workers extract text, then call external AI for summarization and topic classification. Results go to MongoDB, Redis caches hot items; Prometheus and Logstash provide observability.	Frontend: Submit URLs and view summaries. Node.js + Express: JWT-secured API and task creation. Redis: Job status and content cache. Celery (Python): Scraping + NLP summarization tasks. Web Scraping Libraries: Requests + BeautifulSoup/readability to extract content. MongoDB: Stores articles, summaries, and topics. Prometheus: Scrape duration, summarization latency, failure counts. Logstash: Aggregates logs for scraping and AI calls. External AI APIs (examples): Hugging Face: facebook/bart-large-cnn — Abstractive text summarization endpoint. Hugging Face: facebook/bart-large-mnli — Zero-shot topic classification endpoint.
Smart_sales	Develop a web-based platform for sales analysis and prediction using Machine Learning (ML) (max 2 groups)	React,Redis,celery,Flask/FastAPI
LigBind AI	Develop a web-based platform for Drug Design using Machine Learning (ML) (max 2 groups)	React,Redis,celery,Flask/FastAPI
MedZoom AI	Development of a Web-Based Zoom Platform for Medical Image Analysis Using Deep Learning(max 2 groups)	websocket,React,Flask/FastAPI
StreamWatch	Develop a real-time database monitoring system for unstructured data using intelligent analytics and visualization techniques.(max 2 groups)	Prometheus,MongoDB,React,Flask/FastAPI
PhenoPredict	Develop a web-based system for predicting plant phenotypes from genomic data using machine learning techniques.(max 2 groups)	React,Redis,celery,Flask/FastAPI
AskLaw – Smart Legal Q&A with Context Memory and System Insights	Build a full-stack web application that answers everyday legal questions in plain language using an API-based Large Language Model (LLM). Users can type a question or paste a clause, and the system retrieves short, relevant context snippets from a curated legal FAQ and definitions dataset stored in a vector database. These snippets are used to ground the LLM's response, which returns a structured and easy-to-read explanation — outlining What it means, Possible options, Red flags, and Next steps — along with a visible disclaimer stating that it is for educational purposes only, not legal advice. Each user securely logs in through a token-based authentication mechanism, ensuring protected access to personal history, feedback forms, and recommendations. When a user submits a new question, the system first checks an in-memory caching layer for previously generated results to reduce latency and repeated model calls. If no cached response is found, the query is sent to a background task manager that retrieves relevant context, invokes the LLM service, and stores the result along with user feedback in the main database. All user interactions, task events, and system activities are collected through a centralized logging pipeline, providing better traceability and debugging support. Simultaneously, system performance metrics such as response time, cache efficiency, and task queue length are tracked by a monitoring and metrics system, offering real-time visibility into the application's health and performance. By combining these components, AskLaw showcases how intelligent caching, asynchronous task execution, secure authentication, and observability tools can improve the responsiveness and reliability of AI-driven web systems.	Frontend: HTML, CSS, JS Backend: Node.js + Express Database Layer: MongoDB (main) + ChromaDB (vector store) Performance Layer: Redis (cache) + Celery (async tasks) Security Layer: JWT (authentication) Observability Layer: Logstash (logging) + Prometheus (monitoring) LLM Layer: Hugging Face or equivalent AI inference API
TaskFusion: AI-Powered Asynchronous Processing Platform	In this project, you will build TaskFusion, a full-stack web application that demonstrates how modern systems efficiently handle multiple AI-powered tasks in the background without blocking the user experience. Users will log in through a secure authentication system and submit text inputs for processing — such as generating summaries, extracting keywords, or creating detailed reports. Once a user submits a request, the system immediately acknowledges it and queues the task for background processing, allowing the user to continue interacting with the application without waiting for completion. Each submitted request is temporarily stored in memory for faster access and assigned to a background job queue. The queued tasks are processed asynchronously by background workers that execute AI operations (for example, summarization or keyword extraction) using external model APIs. When the output is ready, it is saved in the main database and made available for users to view in their dashboard. The system captures detailed logs of every event — including user requests, task creation, processing status, and completion outcomes — in a centralized logging pipeline, enabling developers to trace errors and analyze usage patterns. It also continuously monitors performance metrics such as queue size, task completion time, API latency, and resource usage, which can be viewed through a live monitoring dashboard. To enhance scalability and reliability, the system uses a robust message broker to distribute background tasks efficiently. A search and analytics layer can optionally be added to allow users or administrators to search across all stored outputs, analyze text content, and visualize processing statistics in real time.	Frontend: HTML, CSS, JavaScript Backend: Node.js + Express, FastAPI Database: MongoDB Caching: Redis Task Queue & Message Broker: Celery and RabbitMQ Authentication: JWT (JSON Web Token) Logging: Logstash Monitoring: Prometheus Search & Analytics (optional): ElasticSearch

LiveQuiz+ — Real-Time Interactive Quiz Platform with Performance and Monitoring Features	In this project, you will build LiveQuiz+, a web-based application that allows teachers to host live quizzes or polls while students join and participate in real time. The teacher can start a quiz, send questions, and immediately see who has responded. Students see new questions appear instantly and can submit answers without refreshing their browsers. Each user securely logs in before joining or hosting a quiz. The system maintains active sessions for all participants and quickly updates results, scores, and leaderboards. To enhance speed and scalability, frequently accessed data like quiz questions, user sessions, and leaderboards are stored in temporary memory for faster access. When a quiz is started or when results need to be compiled, these longer operations are handled in the background to ensure that the main application remains responsive for all users. The system continuously records important events such as quiz creation, user logins, responses, and performance data for analysis and debugging. In addition, a live monitoring dashboard tracks key performance metrics such as response times, background task performance, and overall system health, providing visibility into how the platform performs during heavy usage. Optionally, an AI feature can be added to automatically generate quiz questions or provide real-time feedback based on student performance.	Frontend: HTML, CSS, JavaScript Backend Services: Node.js + Express, FastAPI Database: MongoDB Caching: Redis Task Queue: Celery Authentication: JWT Logging: Logstash Monitoring: Prometheus
RapidRide — Real-Time Ride-Hailing Platform	RapidRide is a scalable, low-latency ride-hailing platform built on the MERN stack (MongoDB, Express, React, Node.js) that connects riders with nearby drivers in real time. The system focuses on resilient real-time communication (location updates, ride matching), reliable background processing (fare calc, receipts, notifications), and operational observability (Logstash/Elasticsearch/Kibana + Prometheus/Grafana). Redis is used for session management, caching, and ephemeral location/pub-sub. Celery handles asynchronous tasks and scheduled jobs; JWT secures user/driver APIs. The platform supports extensible integration points (optional RabbitMQ for advanced messaging, FastAPI for Python microservices, WebSockets for low-latency streams, and ElasticSearch for fast search/analytics). RapidRide aims to deliver a production-grade MVP that's easy to iterate on and scal	Recommended Technology Stack Core (must-have) • Frontend: React.js (Vite or CRA) • Backend: Node.js + Express.js (REST + socket endpoints) • Database: MongoDB (users, trips, payments, ratings) • Cache / Sessions / Pub-Sub: Redis (session store, caching, geo queries, pub/sub) • Background Jobs: Celery (Python workers) — broker: Redis (or RabbitMQ if you need advanced routing) • Auth: JWT (access + refresh tokens) • Logging: Winston → Logstash → Elasticsearch → Kibana (ELK) • Monitoring: Prometheus + Grafana
SocioSphere — Social Media Platform	SocioSphere is a modern, full-stack social media platform built on the MERN stack (MongoDB, Express.js, React.js, Node.js) that enables users to connect, share, and interact in real-time. The system focuses on performance, scalability, and observability, leveraging advanced tools like Redis for session management and caching, Celery for asynchronous task execution, and JWT for secure user authentication. The platform's backend is designed for high-concurrency event handling with structured logging (Logstash + Elasticsearch + Kibana) and metrics monitoring (Prometheus + Grafana) to ensure seamless performance and operational visibility. Key social media functionalities such as posting, liking, commenting, following, and messaging are combined with intelligent caching and background task processing to deliver a responsive and fault-tolerant experience. Optionally, RabbitMQ can be used as a message broker for real-time notifications and activity feeds, while WebSockets may power instant chat and live feed updates. ElasticSearch can optionally enhance content discovery through powerful full-text search and recommendation indexing. SocioSphere thus provides a robust, observable, and scalable social ecosystem, balancing engaging user experiences with enterprise-grade backend architecture.	💻 Technology Stack Frontend (React.js) •React.js– Component-based UI •Redux Toolkit / React Query – State management •Axios / Fetch – API communication •Socket.IO client (optional) – Real-time updates Backend (Node.js + Express.js) •Express.js – REST API framework •MongoDB + Mongoose – Database for users, posts, comments •Redis – Session store & cache •Celery (Python) – Background worker for async tasks •JWT (jsonwebtoken) – Authentication •Logstash + Elasticsearch + Kibana – Logging & visualization •Prometheus + Grafana – Metrics monitoring •Optional: RabbitMQ (message broker), FastAPI (microservice), WebSockets (real-time chat)
SmartEats - On-Demand Food Delivery Application	The rapid digital transformation in the food service industry has increased the need for efficient and intelligent food delivery systems capable of handling large-scale, real-time operations. SmartEats is an advanced On-Demand Food Delivery Platform developed using the MERN (MongoDB, Express, React, Node.js) stack, designed to seamlessly connect customers, restaurants, and delivery partners through a unified, interactive ecosystem. The application leverages Redis for high-speed caching and session management to ensure minimal latency and quick retrieval of user data, menus, and order details. Celery is integrated as a distributed task queue system to manage asynchronous operations such as order dispatching, notifications, and payment confirmations efficiently. Secure and stateless user authentication is achieved through JSON Web Tokens (JWT), allowing smooth authorization across multiple services. For observability and reliability, Logstash (ELK Stack) is employed for centralized logging, enabling streamlined tracking of system logs and real-time error analysis. Prometheus is used to collect and monitor performance metrics, while Grafana provides visual dashboards and alerting mechanisms for operational insights and fault tolerance. Optional enhancements such as RabbitMQ for reliable inter-service communication, FastAPI for lightweight microservice integration, WebSockets for real-time order tracking, Web Scraping for automated data collection, and Elasticsearch for advanced search capabilities further enrich the platform's scalability and intelligence. By combining these technologies, SmartEats establishes a robust, fault-tolerant, and performance-oriented architecture that enhances user experience, optimizes system efficiency, and ensures real-time responsiveness — positioning it as a next-generation solution for on-demand food delivery systems.	Technology Stack: Frontend: React.js, HTML5, CSS3, Bootstrap, Axios Backend: Node.js, Express.js Database: MongoDB, Mongoose Caching & Sessions: Redis Background Processing: Celery (Python) Authentication: JWT (JSON Web Token) Monitoring & Logging: Logstash (ELK Stack), Prometheus, Grafana Real-time Updates (Optional): WebSockets / Socket.IO Microservices (Optional): RabbitMQ, FastAPI Deployment Tools: Docker, Nginx
MarketHub — Online Marketplace & Price Tracker	In today's dynamic e-commerce ecosystem, online marketplaces experience continuous fluctuations in product prices, demand, and sales performance. MarketHub is a comprehensive E-commerce Analytics and Price Tracker Web Application built using the MERN Stack (MongoDB, Express.js, React.js, Node.js), aimed at empowering sellers, buyers, and analysts with actionable business insights. The system provides real-time analytics, product performance tracking, and price variation monitoring across multiple online platforms. The platform leverages Redis for efficient session management and caching, ensuring fast and consistent data delivery. Celery is integrated for asynchronous task scheduling such as periodic data aggregation, background analytics computation, and scheduled price tracking. User authentication and secure communication are implemented using JWT (JSON Web Tokens). For advanced monitoring and performance metrics, Prometheus and Grafana are utilized to provide system-level and application-level dashboards, offering observability and alerting features. The ELK Stack (Elasticsearch, Logstash, Kibana) enhances log management and centralized analytics, enabling developers and administrators to visualize request patterns, detect anomalies, and maintain system reliability. Optionally, RabbitMQ can be incorporated for distributed message brokering to manage communication between analytics microservices, and FastAPI may be used to expose high-performance endpoints for data processing. Additionally, WebSockets can support real-time price updates, while web scraping modules can be employed to fetch live market data from popular e-commerce platforms.	Technology Stack: Frontend: React.js, HTML5, CSS3, Bootstrap, Axios Backend: Node.js, Express.js Database: MongoDB, Mongoose Caching & Sessions: Redis Background Processing: Celery (Python), RabbitMQ (optional) Authentication: JWT (JSON Web Token) Logging & Monitoring: Logstash (ELK Stack), Prometheus, Grafana Web Scraping: Scrapy / Puppeteer Real-time Updates: Socket.IO / WebSockets Deployment Tools: Docker, Nginx (optional)

Smart Movie Booking and Management System	The Smart Movie Booking and Management System is a full-stack web application built using the MERN stack (MongoDB, Express.js, React, Node.js) that enables users to seamlessly browse, book, and manage movie tickets across multiple theaters in real time. The system allows users to explore movie listings, view showtimes, select preferred seats, and make secure online bookings with instant confirmation. To enhance scalability and performance, the backend integrates Redis for session management and caching frequently accessed data such as movie details, showtimes, and user sessions. Celery (with a message broker like RabbitMQ) handles asynchronous background tasks, including booking confirmations, email notifications, and analytics updates, ensuring that the user-facing API remains fast and responsive during peak loads. The React.js frontend provides an interactive, user-friendly interface that includes dynamic seat selection, movie trailers, user reviews, and real-time availability updates. The backend, powered by Node.js and Express.js, manages user authentication via JWT, booking logic, and communication with the MongoDB database, which stores user profiles, theater information, and booking records. An optional Streamlit-based Admin Dashboard allows theater administrators to visualize booking trends, track occupancy rates, and manage movie schedules in real time. By combining asynchronous processing, caching, and modern web technologies, the system delivers a reliable and scalable solution for movie ticket booking and management—ideal for beginners learning distributed systems and full-stack development.	Technology Stack Frontend: React.js, HTML5, CSS3, Bootstrap Backend: Node.js, Express.js. Database: MongoDB, Mongoose Caching & Session Management: Redis Background Processing: Celery (Python) RabbitMQ (optional) — as the message broker to queue background jobs efficiently Authentication & Security: JWT bcrypt.js — for password hashing Real-time Updates: Socket.IO / WebSockets Logging & Monitoring: Logstash (ELK Stack) Prometheus — for performance metrics and health monitoring Deployment & DevOps Tools: Docker
Smart Campus Portal: An AI-Driven Integrated Web Platform Using ReactJS, Flask, Express, and MongoDB"	Build a unified Smart Campus Portal where students, faculty, and administrators can securely access academic services through a single platform. Users interact through a ReactJS dashboard, while authentication and core data operations are managed by an Express.js backend backed by MongoDB. AI-driven features such as chatbot assistance, announcement summarization, and performance recommendations are handled asynchronously: Celery workers call a Flask or FastAPI-based AI microservice that may use external NLP models (e.g., Hugging Face or OpenAI).	Technology Stack Frontend: React.js, HTML5, CSS3 Backend: Node.js, Express.js Database: MongoDB. Caching & Session Management: Redis,FastAPI Background Processing: Celery (Python) , RabbitMQ (optional) Authentication & Security: JWT (JSON Web Token) Real-Time Updates: Socket.IO Logging & Monitoring: Logstash (ELK Stack), Prometheus Admin Dashboard (Optional): Streamlit Deployment & DevOps Tools: Docker
SocialSync – Scalable Social Media Clone using ReactJS, FastAPI, Redis, RabbitMQ, Celery, Theobroma, Logstash, Graphini, and Socket.IO	SocialSync aims to design and develop a social media application clone that demonstrates real-time performance, scalability, and intelligent monitoring using a microservice-based architecture.The system will support user authentication, post creation, live feeds, notifications, chats, and analytics dashboards, leveraging cutting-edge backend technologies such as FastAPI, Redis, RabbitMQ, and Celery. Logstash and Graphini will be used for observability and data visualization, while Theobroma will coordinate service health and communication	Technology Stack Frontend: React.js, HTML5 Backend: Node.js, Express.js Database: MongoDB, Mongoose . Caching & Session Management: Redis,FastAPI Background Processing: Celery (Python) , RabbitMQ (optional) Authentication & Security: JWT (JSON Web Token) Real-Time Updates: Socket.IO / WebSockets Logging & Monitoring: Logstash (ELK Stack), Prometheus Admin Dashboard (Optional): Streamlit
Smart Real-Time Job Aggregator and Recommendation System	Finding suitable jobs efficiently is challenging due to fragmented job portals, duplicate postings, slow manual searching, and lack of personalization. This system aims to provide a centralized, intelligent, and real-time job aggregation system that delivers personalized recommendations and notifies users instantly of matching openings.	
Real-Time News Summarization & Sentiment Analysis Portal	This project develops a real-time web application that collects, summarizes, and analyzes news articles from multiple online sources using Natural Language Processing (NLP). The system scrapes content, generates concise summaries, and performs sentiment analysis (positive, negative, neutral). It delivers live updates via WebSocket, uses Redis caching for trending news, and integrates Prometheus and Logstash for performance monitoring and logging. Secure authentication with JWT allows users to personalize their feeds and bookmarks	
Real-Time Vehicle Tracking and Fleet Management System	The Real-Time Vehicle Tracking and Fleet Management System is a web-based platform designed to monitor the live location, performance, and usage of vehicles in a fleet. It provides logistics companies and drivers with instant updates about vehicle routes, fuel usage, engine status, and delivery performance. Using IoT-enabled GPS devices, vehicle data such as speed, location, and route deviation is sent to the backend in real-time. Redis is used for caching frequently changing GPS coordinates to ensure faster updates on the dashboard, while RabbitMQ or Celery manages background tasks like sending delay alerts, route optimizations, or maintenance reminders. The React.js-based dashboard displays all vehicles on an interactive live map, with color-coded markers showing real-time statuses. JWT ensures secure authentication for admins and drivers, while Prometheus and Logstash monitor system performance and logs. The system enables logistics operators to improve fleet efficiency, reduce operational costs, and make data-driven decisions using live tracking and analytics.	Frontend: HTML, CSS, JavaScript, React.js Backend: Node.js, Express.js Database: MongoDB Middleware: Redis, RabbitMQ, Celery Security: JWT Monitoring: Prometheus, Logstash Real-Time Updates: WebSocket Search: ElasticSearch

Real-Time Health Monitoring System	The Real-Time Health Monitoring System is a web-based platform designed to continuously track, analyze, and visualize patients' vital health parameters such as heart rate, blood pressure, oxygen saturation, and body temperature. The system collects health data from IoT-enabled devices or manual input and stores it securely in a MongoDB database. The React.js-based dashboard provides real-time visualization of patient data with interactive charts and alerts. Redis is used for caching frequently accessed health data, ensuring faster response times, while RabbitMQ/Celery handle background tasks like sending alerts, generating reports, or notifying doctors asynchronously. JWT (JSON Web Token) ensures secure authentication for patients and healthcare providers. Prometheus and Logstash monitor the application's performance and maintain system logs, while WebSocket provides live data updates to the dashboard. The system helps doctors and patients detect abnormalities early and take immediate medical actions.	Frontend: HTML, CSS, JavaScript, React.js Backend: Node.js, Express.js Database: MongoDB Middleware: Redis, RabbitMQ, Celery Security: JWT Monitoring: Prometheus, Logstash Real-Time Updates: WebSocket Search: ElasticSearch
AI Knowledge Based/Doc Assistant using RAG Model	This project develops an AI-powered Knowledge Assistant that retrieves, understands, and answers user queries using enterprise documents. It integrates a Retrieval-Augmented Generation (RAG) pipeline built with FastAPI, combining the power of vector-based semantic search and large language models (LLMs). Redis is used for caching query embeddings and session data to improve response speed. RabbitMQ and Celery manage asynchronous document ingestion and model inference tasks. Logstash provides centralized logging and system monitoring, while Prometheus gathers performance metrics for observability. Secure access control is handled through JWT-based authentication. This architecture ensures scalable, real-time AI responses with fault tolerance and transparency, making it ideal for corporate knowledge bases, healthcare documentation systems, or academic repositories.	
AI-Based Personal Trainer Assistant	A backend system that records gym member workouts, tracks progress, and analyzes performance trends. Redis caches real-time workout stats. Celery + RabbitMQ handle background job processing (e.g., calculating calories burned). Logstash captures user activity and API logs. Prometheus monitors API uptime and performance. Feature Highlights: JWT-secured login for trainers & members AI-based suggestion system (via async Celery task) Weekly progress reports sent via RabbitMQ queue Tech Stack:ReactJS, FastAPI, Redis, Celery, RabbitMQ, JWT, Prometheus, Logstash	
Online Healthcare Appointment & Pharmacy System	The MediConnect project aims to develop a web-based healthcare platform that enables patients to conveniently book appointments with doctors, consult online, and purchase prescribed medicines through an integrated pharmacy system. The system bridges the gap between healthcare providers and patients by digitizing medical interactions and simplifying access to health services Patients can search for doctors based on specialization, view availability, and schedule appointments online. The integrated pharmacy module allows users to upload prescriptions and place orders for medicines, which are verified by pharmacists before dispatch. Doctors can manage their schedules, view patient records, and offer consultation feedback. The system ensures secure storage and retrieval of sensitive medical data using encrypted communication between the frontend and backend. The project leverages the MERN stack (MongoDB, Express.js, React.js, Node.js) for fast, scalable, and responsive development. Middleware such as JWT authentication and RESTful APIs ensure secure and modular integration of components. MediConnect provides a unified platform promoting accessibility, efficiency, and transparency in healthcare delivery.	
A Full-Stack Hotel Booking Ecosystem	This project outlines the design and development of "Horizon Haven," a sophisticated, cloud-ready hotel and room booking platform. The primary objective is to create a seamless, responsive, and highly available web application that enhances the user experience for both guests and hotel administrators. By integrating a modern MERN technology stack with robust backend services like Redis, Celery, and advanced monitoring tools, the platform addresses critical challenges in the hospitality industry, including real-time room availability, secure transactions, performance bottlenecks, and efficient background task processing.	• Frontend: React.js • Backend: Node.js • Database: MongoDB • Authentication & Authorization: JSON Web Tokens (JWT) • Redis for Caching and Real-time Performance • Celery for Asynchronous Task Management • Logstash for Centralized Logging • Prometheus & Grafana for Monitoring and Visualization:
A Scalable E-Commerce Platform for Organic Agri-Products	This project outlines the design and development of "GreenHarvest," a sophisticated, cloud-native e-commerce platform dedicated to the sale of certified organic agricultural products. The primary objective is to create a transparent, efficient, and high-performance marketplace that directly connects local farmers with health-conscious consumers. By leveraging the MERN stack integrated with robust backend services like Redis, Celery, and advanced observability tools, the platform addresses key challenges in the organic supply chain, including real-time inventory management, supply chain traceability, secure transactions, and processing of resource-intensive tasks.	• Frontend: React.js • Backend: Node.js • Database: MongoDB • Authentication & Authorization: JSON Web Tokens (JWT) • Redis for Caching, Session Management & Real-time Inventory • Celery for Asynchronous Task Management • Logstash for Centralized Logging and Agri-Data Pipeline: • Prometheus & Grafana for Performance Monitoring and Business Intelligence:
Real Time Data Flow Banking Sector Architecture	The main aim of the Real-Time Data Flow Banking Sector Architecture project is to design and develop a full-stack, event-driven banking platform capable of processing multiple financial operations in real time using asynchronous background task execution. The project focuses on improving transaction speed, reliability, fault tolerance, and data synchronization across distributed banking systems. It aims to demonstrate how modern banking platforms can leverage software design patterns, middleware layers, and asynchronous communication mechanisms to manage large-scale financial workflows seamlessly, ensuring high availability, low latency, and consistent data integrity across all transactions. In essence, the project bridges the gap between traditional synchronous banking operations and modern event-driven architectures, creating a secure, modular, and highly scalable environment for handling complex banking activities such as payments, transfers, and reporting.	Technologies: React,WebSockets,FastAPI,Celery,JWT,PostgreSQL,MongoDB, RabbitMQ,Redis,Logstash,Elasticsearch

Major Diseases Design Pattern Architecture	In this project, you will build MediFusion, a full-stack AI-powered disease detection and management web platform that demonstrates how modern healthcare systems efficiently process multiple diagnostic and medical analytics tasks in the background — without interrupting the user experience. Healthcare professionals require a unified, intelligent platform that can handle multiple disease detection and analytics tasks simultaneously, while ensuring that users (doctors, lab staff, or patients) can continue interacting with the system seamlessly. The problem MediFusion solves is how to integrate AI and asynchronous background task processing into a unified, real-time disease detection and management platform, ensuring speed, scalability, and reliability across all operations. The main aim of the MediFusion project is to design and develop a full-stack, AI-powered, and event-driven disease detection and management platform that enables real-time healthcare operations through asynchronous processing and intelligent automation. The system aims to integrate AI-based diagnostic models, secure APIs, and scalable middleware technologies to ensure that multiple medical analytics tasks can be processed efficiently in the background — without interrupting or degrading the user experience. In essence, the project seeks to bridge the gap between real-time healthcare interfaces and asynchronous AI-driven diagnosis, providing an efficient, transparent, and scalable medical data processing environment for hospitals, laboratories, and telemedicine platforms.	Technologies: React, FastAPI, Redis,celery, JWT,RabbitMQ, PostgreSQL, Hugging Face, ChromaDB,PyTorch, Gemini API, Singleton,Elasticsearch.
Development of a Web-Based Social Media Platform	The Web-Based Social Media Platform is an interactive online system that enables users to create profiles, share multimedia content, communicate with others, and form online communities. The system focuses on real-time interaction, data privacy, and content personalization through advanced backend technologies and scalable infrastructure.	Frontend: HTML,React.js,CSS, Prometheus and Grafana. Backend: FastAPI or Node.js,PostgreSQL,Redis,Celery and message queues.
Develop web based application for Farmer Advisory and Decision-Support System	The Farmer Advisory and Decision-Support System (FADSS) is a web-based platform designed to help farmers make data-driven agricultural decisions regarding crop selection, irrigation scheduling, fertilizer usage, and pest management. The system integrates data from multiple sources — such as weather forecasts, soil parameters, and market trends — and uses machine learning and rule-based analytics to generate personalized recommendations for farmers.	Frontend: HTML,React.js,CSS, Prometheus and Grafana. Backend: FastAPI or Node.js,PostgreSQL,Redis,Celery and message queues.
IOT BASED SMART Electricity Conservation system	With the growing global demand for electricity, efficient energy management has become a critical challenge. Conventional electrical systems often lack real-time monitoring, automation, and intelligent control, resulting in significant power wastage and increased operational costs. The proposed IoT-based Smart Electricity Conservation System aims to address this problem by integrating sensors, microcontrollers, and cloud-based data analytics to track, analyze, and manage power usage in real time. Through automated control and user feedback mechanisms, the system seeks to minimize energy wastage, lower electricity costs, and contribute to sustainable energy utilization.	Hardware:microcontroller and Sensor Software:Arduino IDE / MicroPython – device programming IoT Platform: Blynk / ThingSpeak / Firebase Database: Firebase / MySQL Frontend: HTML, CSS, JavaScript / Blynk App – user interface Communication:Wi-Fi, MQTT / HTTP protocols for data transfer
Smart Campus Facility and Maintenance Management System	Develop a centralized Smart Campus Maintenance System where:Students/faculty can report issues online.Maintenance staff can view and update assigned requests.Admins can monitor request status, team efficiency, and trends in real-time	FrontEnd : HTML, CSS, JavaScript BackEnd: Node.js + Express, FastAPI Database : MongoDB Caching : Redis Task Queue : Celery Authentication : JWT Logging : Logstash Monitoring : Prometheus
Smart Task & Escalation Management System	The Smart Task & Escalation Management System (STEMS) aims to automate the task assignment, reminder, and escalation process by integrating intelligent backend automation and performance monitoring. STEMS integrates modern full-stack, asynchronous, and monitoring technologies to simulate how enterprise systems handle distributed workloads and maintain operational observability.	FrontendReact.js BackendNode.js + Express.js DatabaseMongoDB AuthenticationJWT Cache / QueueRedis WorkerCelery LoggingLogstash MonitoringPrometheus
E-Commerce Order Tracking and Analytics System	Modern e-commerce businesses handle thousands of concurrent orders, often spread across multiple warehouses, courier services, and regions. Tracking every order's status — from payment processing → dispatch → shipment → delivery — becomes challenging, especially when dealing with delays, failed transactions, or system slowdowns. The goal is to build a centralized dashboard that provides real-time visibility, automated analytics, and proactive alerts for delayed or failed deliveries.	FrontendReact.js + Ant Design BackendNode.js + Express DatabaseMongoDB AuthenticationJWT Caching LayerRedis Async Task QueueCelery (Python) Logging LayerLogstash MonitoringPrometheus + Grafana Real-time UpdatesWebSockets (Socket.io)
Virtual Mental Health Companion	Mental health support is expensive and inaccessible for many. A virtual AI companion can offer early intervention and emotional tracking.	NLP (Transformers), MERN, JWT Redis, WebSocket, FastAPI, Prometheus
Online Dispute Resolution Platform	Minor civil disputes take years to resolve; a digital arbitration platform can speed up resolution	MERN, JWT, WebSocket FastAPI, Redis, Prometheus
Predictive Cyber Defence System	Defence networks face advanced cyber attacks; predictive AI models can detect anomalies ea	Python, FastAPI, Prometheus, JWT Elasticsearch, Redis, Logstash
AI-powered Radiology Workflow Optimizer	Radiologists face high workloads, and often bottlenecks delaying diagnosis. An AI tool that automates parts of workflow (e.g., triage scans, highlight urgent ones) can reduce delay.	Image processing (TensorFlow or PyTorch) for scan analysis, FastAPI service to expose results, MongoDB for patient/scan metadata, JWT for secure access. Redis for caching results, Elasticsearch for indexing reports, Celery for batch processing, Logstash for logging workflow metrics.
Evidence & Proceeding Analytics Platform for Courts	Courts generate huge amounts of data (proceedings transcripts, filings, judgments). Analytics on evidence patterns, judge behaviour, typical delays can empower better case strategy and governance.	Database (MongoDB or PostgreSQL) for court data, backend (FastAPI/Node.js), JWT for secure access. NLP to process transcripts, Elasticsearch for search & query, Redis for fast lookups, WebSocket for dashboards live updates, data-visualisation library (D3.js) for insights.
A Real-Time Medicine Availability & Delivery System	Patients often face difficulty in finding essential medicines during emergencies, especially at night or in remote areas. Many pharmacies do not have updated stock information, leading to delays and inconvenience for patients in urgent need. To solve this, there is a need for a system that can manage live medicine data, ensure secure access, monitor performance, and provide efficient background processing for orders and updates.	Frontend (User Interface) - HTML, CSS, JavaScript Backend (Core Logic & APIs) - FastAPI (Python) Authentication JWT - (JSON Web Token) Database / Caching Layer - Redis Search Engine - Elasticsearch Background Task Queue - Celery Monitoring & Metrics - Prometheus Logging & Analytics - Log Stats (Logging System)